

NMPDE project presentation

Stefano Mancini

September 3, 2024

This project is based on Step 26 of the deal.II tutorial. It has been set up to work with the *bare-dealii-app* template by prof. Luca Heltai. Instead of using Rothe's method, we will use the method of lines to solve the heat equation. The boundary conditions will be imposed via Nitsche's method. For the integration in time, we will rely on the ARKode package that is part of the SUNDIALS suite. Adaptive mesh refinement will be implemented via solution transfer.

1 OVERVIEW OF THE PROBLEM

We consider the heat equation:

$$\begin{cases} u_t(x, t) - \Delta u(x, t) = f(x, t) & (x, t) \in \Omega \times (t_0, t_1] \\ u(x, t_0) = u_0(x) & x \in \Omega \\ u(x, t) = g(x, t) & (x, t) \in \partial\Omega \times [t_0, t_1] \end{cases}$$

In particular, the domain Ω is a `hyper_L` (2D and 3D case) or the interval $[-1, 1]$ (1D case). Before we start, we need to address the following issues:

- How do we discretize the PDE?
- What weak form do we choose?

To solve the PDE numerically, we use semi-discretization. In particular, there are two ways to do this:

- *Rothe's method*, i.e. discretize in time first and get a PDE which is only space-dependent; then treat the resulting problem with a finite element method. This means that the time stepping method will be pre-fixed at start.
- The *method of lines*, i.e. discretize in space first and get an ODE that depends only on time; then solve the ODE with an IVP solver.

Each way has its advantages. We choose the method of lines because Rothe's method requires us to write the time stepping by hand to get the problem that will be solved with a finite element method. This can be painful, especially when using higher order formulas or if we plan to try different time steppings. On the other hand, professional libraries for ODE solution, such as SUNDIALS, save us this effort and also implement strategies for time step adaptivity.

Regarding the weak form: a standard way to solve stationary problems is to split the solution u into a sum $u_0 + u_g$, such that u_0 satisfies homogeneous boundary conditions and $u_g = g$ at the boundary (in terms of trace). Then we solve for $u_0 \in H_0^1(\Omega)$. We can do the same for time dependent problems, but this approach has a main disadvantage: the boundary condition g changes over time. Thus, we use Nitsche's method and incorporate the boundary condition directly in the weak form.

2 WEAK FORM

The weak form we obtain by using Nitsche's method is the following:

$$(u_t, v) + (\nabla u, \nabla v) - \langle \nabla u \cdot n, v \rangle - \langle u, \nabla v \cdot n \rangle + \gamma \langle u, v \rangle = (f, v) - \langle g, \nabla v \cdot n \rangle + \gamma \langle g, v \rangle$$

where $(\cdot, \cdot)$ is the inner product in Ω , $\langle \cdot, \cdot \rangle$ the inner product in $\Gamma = \partial\Omega$ and n is the normal to Γ . The parameter $\gamma > 0$ is fixed: we know that it has to satisfy some constraint in order for the method to work well.

Let V_h be our FE space (we use Lagrange elements) and let φ_i be the basis functions. We assume that the numerical solution $u_h \in V_h$ can be written as $u_h(x, t) = \sum U_i(t) \varphi_i(x)$, i.e. the degrees of freedom are time dependent. This leads to solving the following ODE:

$$M \mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u})$$

where

$$\begin{aligned} \mathbf{u}_i &= U_i(t) \\ M_{ij} &= (\varphi_i, \varphi_j) \\ f_E(t, \mathbf{u})_i &= (f(\cdot, t), \varphi_i) + \gamma \langle g(\cdot, t), \varphi_i \rangle - \langle g(\cdot, t), \nabla \varphi_i \cdot n \rangle \\ f_I(t, \mathbf{u}) &= J \mathbf{u} \\ J_{ij} &= -(\nabla \varphi_i, \nabla \varphi_j) + \langle \varphi_i, \nabla \varphi_j \cdot n \rangle + \langle \nabla \varphi_i \cdot n, \varphi_j \rangle - \gamma \langle \varphi_i, \varphi_j \rangle \end{aligned}$$

In particular, it is clear that the matrices M and J are independent of time, therefore they need to be evaluated only one time (and re-evaluated only when the mesh is changed).

3 SOLVING THE ODE

To determine the evolution of the degrees of freedom, we rely on the ARKode package of the SUNDIALS suite, which solves IVPs in $\mathbb{R}^n$ with a variety of variable-step, embedded, additive Runge-Kutta solvers (ARK). Generally speaking, consider the IVP

$$\begin{cases} M(t)y' = f_E(t, y) + f_I(t, y), & t \in [t_0, T] \\ y(t_0) = y_0 \end{cases}$$

where $M(t)$ is a nonsingular matrix for every t . In order to solve it, ARKode uses an s -stage additive Runge-Kutta method of the form:

$$\begin{aligned} z_i &= y_{n-1} + h_n \sum_{j=1}^{i-1} A_{i,j}^E \hat{f}_E(t_{n,j}^E, z_j) + h_n \sum_{j=1}^i A_{i,j}^I \hat{f}_I(t_{n,j}^I, z_j), \quad i = 1, \dots, s, \\ y_n &= y_{n-1} + h_n \sum_{i=1}^s \left(b_i^E \hat{f}_E(t_{n,i}^E, z_i) + b_i^I \hat{f}_I(t_{n,i}^I, z_i) \right), \\ \tilde{y}_n &= y_{n-1} + h_n \sum_{i=1}^s \left(\tilde{b}_i^E \hat{f}_E(t_{n,i}^E, z_i) + \tilde{b}_i^I \hat{f}_I(t_{n,i}^I, z_i) \right), \end{aligned}$$

where we have set $\hat{f}_E(t, y) = M(t)^{-1} f_E(t, y)$ and $\hat{f}_I(t, y) = M(t)^{-1} f_I(t, y)$. In particular:

- $\tilde{y}_n$ are embedded solutions that are used for error estimation; these typically have slightly lower accuracy than the computed solutions y_n .
- The internal stage times of the method are $t_{n,j}^E = t_{n-1} + c_j^E h_n$ and $t_{n,j}^I = t_{n-1} + c_j^I h_n$.
- An explicit method and an implicit one are used at the same time and share the same number s of stages. The coefficients from their Butcher's tables are $A^E \in \mathbb{R}^{s \times s}$, $b^E \in \mathbb{R}^s$ and $\tilde{b}^E \in \mathbb{R}^s$, for the explicit method, and their counterparts with the I , for the implicit one. Finally, the coefficients $\tilde{b}^E \in \mathbb{R}^s$ and $\tilde{b}^I \in \mathbb{R}^s$ are used to construct the embedding.

- The implicit method is diagonally implicit (DIRK).

Let us stress for a moment why we have split the right hand side into a sum of two contributions. When a system presents both fast and slow dynamics, it is important to separate them:

- f_E contains the "slow" (*nonstiff*) time scale components of the system. This part will be integrated using explicit methods.
- f_I contains the "fast" (*stiff*) time scale components of the system. This will be integrated using implicit methods.

Of course we could have treated f_E as part of f_I but, on the other hand, we have to try to optimize the computational effort.

When we use ARK methods, an implicit system of the form

$$G(z_i) := Mz_i - h_n A_{i,i}^I f_I(t_{n,i}^I, z_i) - a_i = 0$$

must be solved for each stage z_i , with $i = 1, \dots, s$, where we have the data

$$a_i := M y_{n-1} + h_n \sum_{j=1}^{i-1} [A_{i,j}^E f_E(t_{n,j}^E, z_j) + A_{i,j}^I f_I(t_{n,j}^I, z_j)].$$

In our specific case, $f_I(t, y)$ will depend linearly on y , then this will be a linear system of equations. This will allow ARKode to take some shortcuts for a faster solution process. In particular, we choose the default solution strategy, i.e. a variant of Newton's method:

$$z_i^{m+1} = z_i^m + \delta^{m+1},$$

where m is the Newton step index. The Newton update δ^{m+1} requires the solution of the linear Newton system

$$N(z_i^m) \delta^{m+1} = -G(z_i^m),$$

where

$$N := M - \gamma J, \quad J := \frac{\partial f_I}{\partial y}, \quad \gamma := h_n A_{i,i}^I.$$

Since $f_I(t, y)$ depends linearly on y , each system will be solved using only a single Newton iteration. Moreover, to assure efficiency and robustness of the algorithm, the initial guess the method will be a predicted value $z_i(0)$ that is computed explicitly from the previously-computed data (e.g. y_{n-2} , y_{n-1} and z_j for $j < i$). Additional information on the specific predictor algorithms implemented in ARKode is provided in ARKode documentation.

In conclusion, let us write again the IVP we will try to solve:

$$\begin{cases} M \mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u}), & t \in [t_0, t_1] \\ \mathbf{u}(t_0) = \mathbf{u}_0 \end{cases}$$

where $\mathbf{u}_0$ is the initial condition $u_0(x)$, interpolated with respect to the basis functions φ_i .

4 DEALING WITH ADAPTIVE MESHES

Adapting the mesh is a key point for time dependent problems: the mesh has to "follow" the solution in some way and predict incoming changes in its behaviour. However, there are significant difficulties compared to the stationary case.

4.1 Time step size and minimal mesh size

Let us cite briefly from the Step 26 documentation. For stationary problems, the general approach is "make the mesh as fine as it is necessary". For problems with singularities, this often leads to situations where we get many levels of refinement into corners or along interfaces.

For time dependent problems, we typically have error estimates of the form

$$\|e\| \leq O(k^p + h^q)$$

where p, q are the convergence orders of the time and space discretization, respectively. We can only make the error small if we decrease both terms. Ideally, an estimate like this would suggest to choose $k \propto h^{q/p}$. Since, at least for problems with non-smooth solutions, the error is typically localized in the cells with the smallest mesh size, we have to indeed choose $k \propto h_{min}^{q/p}$, using the smallest mesh size h_{min} .

Having to choose the time step related to the mesh size is not a negligible issue, because using a significantly smaller time step means having to solve the global linear system more often. This implies a growth of the computational effort, typically bigger than that produced by a slight increase of the number of degrees of freedom.

In practice, having acknowledged that we can not make the time step arbitrarily small, we avoid to make the local mesh size arbitrarily small, too. Rather, we set a maximal level of refinement and, when we flag cells for refinement, we simply do not refine those cells whose children would exceed this maximal level of refinement. In a similar fashion, we want to be ready for a sudden refinement (e.g. when we have a source that switches on in different parts of the domain). Therefore, it is wise to enforce in our program a minimal mesh refinement level.

4.2 Test functions from different meshes

Let us recall the IVP we are solving:

$$\begin{cases} M\mathbf{u}' = f_E(t, \mathbf{u}) + f_I(t, \mathbf{u}), & t \in [t_0, t_1] \\ \mathbf{u}(t_0) = \mathbf{u}_0 \end{cases}$$

Suppose that at some time $t_* > t_0$ we decide to refine the mesh. This leads to a new set of basis functions $\{\hat{\phi}_i\}$, which in turn leads to new matrices $\hat{M}$ and $\hat{J}$, plus new functions $\hat{f}_E$ and $\hat{f}_I$. Moreover, the solution, starting from $t = t_*$, will have to be expressed as a linear combinations of the new basis functions:

$$u(x, t) \approx \sum \hat{U}_i(t) \hat{\phi}_i(x).$$

If $\hat{\mathbf{u}}_i = \hat{U}_i$, the new IVP we have obtained is the following:

$$\begin{cases} \hat{M}\hat{\mathbf{u}}' = \hat{f}_E(t, \hat{\mathbf{u}}) + \hat{f}_I(t, \hat{\mathbf{u}}), & t \in [t_*, t_1] \\ \hat{\mathbf{u}}(t_*) = \hat{\mathbf{u}}_* \end{cases}$$

How do we get the vector $\hat{\mathbf{u}}_*$? In effect, we know the approximated solution at time t_* , but as a linear combination of the previous basis functions. Hence, we have to *transfer* it to the new mesh:

$$\hat{\mathbf{u}}_* = I_{t_*} \mathbf{u}(t_*),$$

where I_t is the interpolation operator onto the finite element space used at time t . Of course this approach introduces an additional error besides time and space discretization, but it is pragmatic and makes it feasible to do time adapting meshes.

4.3 When do we refine?

The last issue is: what kind of *a posteriori* estimate do we use to decide whether a mesh refinement is needed? In our program, we use the Kelly error estimator, which is a simplified version of the *a posteriori* error estimator that is used for the Poisson problem:

$$\eta_T := \sum_{F \in \partial T} \frac{1}{2} h_F^{\frac{1}{2}} \| [\![\nabla u_h]\!] \|_{0,F}.$$

Here u_h represents the approximated solution. We fix some checkpoints $t^{(1)}, t^{(2)}, \dots \in [t_0, t_1]$ and refine the mesh at time $t^{(k)}$ if

$$\sum_{T \in \mathcal{T}_h} \eta_T^2 > \text{tol}$$

for a fixed tolerance `tol`. In particular, we leave room for doing multiple refinements at the same time. This comes in handy, for example, when there is a sudden variation in the solution. However, since the tolerance is fixed a priori, we have to be careful in order to avoid a never ending refinement loop. In fact, one may happen to choose a tolerance that is too low, and our program may not be able to adapt the mesh sufficiently (remember that we have set an inferior limit to the mesh size). Hence, we stop the refinement loop if the number of successive refinements goes beyond a certain threshold. Also we can print the l^2 norm of the error estimator, in order to help the user to tweak the parameters for a better approximation.

5 A FEW COMMENTS ON THE PROGRAM

For each dimension $d = 1, 2, 3$ the program is built in two versions: a debug mode (.g) and a release mode. The program will look for the following directories to write its output and parameters:

- output_1d
- output_2d
- output_3d
- parameters

Make sure to create them in advance in the folder where you will put the executables. Also, in the code there is a `if constexpr` statement, which requires your compiler to support at least C++17.

At first use, a .prm file will be written in the directory `parameters`. This contains all the customizable parameters for the problem. In particular, you can change f , u_0 , g , t_0 and t_1 , along with γ , a variety of parameters for the ARKode solver, the finite element degree and the mesh refinement strategy.

If you want to test a manufactured solution, you can provide the exact solution u in the .prm file: in that case, the program will compute the L^2 error for the numerical solution. Some examples of manufactured solution were provided in the repository for testing, along with a Jupyter Notebook `manufactured_heat.ipynb` to help the user compute the correct f automatically, given u .

A few more technical details:

- The assembly of matrices and vectors is done via a MeshWorker loop.
- Every time the parameter γ is admissible, the linear systems involved in the IVP have a symmetric and positive definite matrix. Therefore, we use the conjugate gradient method to solve them.
- As a preconditioner, we use an algebraic multigrid one (AMG). This is implemented in deal.II with the `TrilinosWrappers::PreconditionAMG` class. For an optimal usage,

it is recommended in the deal.II documentation to use this class in conjunction with Trilinos matrices and vectors. This is what is done in the program, but is not mandatory. In fact, `PreconditionAMG` objects support also `SparseMatrix` and `Vector` objects from deal.II. The main difference is that, in the latter case, a copy of the matrix is generated, which is slower and takes more memory.

- As default, the program checks if the mesh has to be refined at each output step. If you want that check to be more frequent, you can modify the parameter "Error estimations per output period" in the .prm file: the additional checkpoints will be added uniformly between consecutive outputs.

6 SOME TESTS AND RESULTS

We refer to the manufactured functions provided in the file `test_functions.txt`.

6.1 Tests with no adaptive refinement

Two main key points have to be addressed. First, choosing the "right" value of the Nitsche parameter γ is not straightforward:

- If it is too low, then the method becomes unstable.
- Raising γ leads to an increase of computational cost, because ARKode will use a significantly lower timestep size, hence you will have a lot more function evaluations and linear systems to be solved.
- Choosing γ too close to the minimal value that assures stability won't provide you the best accuracy (keeping the other parameters fixed). On the other hand, there's no gain in increasing it *ad libitum*, because there seems to be a trade-off value after which the accuracy lowers. Therefore, if you want maximum accuracy, you may want to reach that trade-off value as a compromise between accuracy and ARKode steps.
- A higher mesh refinement or FE degree do require a higher value of γ to maintain the stability of the method.

Second point, the relation between timestep size and mesh size:

- Increasing the refinement of the mesh leads to a better accuracy, at the cost of more ARKode steps. This makes sense, given the discussion in section 4.1.
- Increasing the FE degree leads also to a better accuracy, *sometimes* with less cost in terms of ARKode steps than increasing the global mesh refinement.
- In general, ARKode adapts the time step automatically to fulfill a desired tolerance. In our program, that tolerance has no relation with the mesh size at all. As a consequence, we may likely get in the situation where either the FEM error or the ARK error prevails on the other, meaning that we are wasting computational resources. Balancing the step size and the mesh size by hand may not be straightforward, but it would be a nice additional feature.
- In some rare cases, strange oscillatory patterns show up in the L^2 error throughout time, even though a certain minimal accuracy is maintained. I conjecture that, again, this may be due to either the FEM error or the ARK error prevailing on the other, partially ruining the convergence order.

Bonus point, which is a small "sanity check": steeper solutions may require finer meshes or higher FE degree, which makes sense.

It hasn't been done here, but it would be interesting to try to tweak also the AMG preconditioner and see if there are significant improvements.

Let us present some data. The 1D case has been tested extensively, as well as the 2D case. The 3D case is more difficult to be tested, because raising the FE degree or the global refinement increases the degrees of freedom by a lot. However, some word will be spent further on. For some screenshots and animations, you can refer to the gallery in the README.

6.1.1 Problem 0.1, 1D case

	Error at final time	ARKode steps	Last stepsize
Default settings	0.0170196	253	0.0103807
$\gamma = 20$	0.0132822	534	0.00485258
$\gamma = 50$	0.0139157	1147	0.000898033
<code>initial_refinement = 5, $\gamma = 10$</code>	explodes		
<code>initial_refinement = 5, $\gamma = 20$</code>	0.00312233	1410	0.0009152
<code>initial_refinement = 5, $\gamma = 50$</code>	0.000916882	2187	0.00141454
<code>initial_refinement = 5, $\gamma = 100$</code>	0.000900294	4605	0.000753134
<code>fe_degree = 2, $\gamma = 10$</code>	explodes		
<code>fe_degree = 2, $\gamma = 20$</code>	9.34488×10^{-7}	680	0.000888443
<code>fe_degree = 2, $\gamma = 50$</code>	1.34329×10^{-5}	1773	0.00235106

6.1.2 Problem 0.2, 1D case

	Error at final time	ARKode steps	Last stepsize
Default settings	0.170172	297	0.0132492
<code>initial_refinement = 5, $\gamma = 100$</code>	0.0090014	4801	0.00107145
<code>fe_degree = 2, $\gamma = 50$</code>	3.19038×10^{-6}	1953	0.00164072
<code>fe_degree = 2, $\gamma = 100$</code>	0.000306053	3747	0.00472022

6.1.3 Problem 1.1, 1D case

	Error at final time	ARKode steps	Last stepsize
Default settings	0.123128	209	0.0143597
<code>initial_refinement = 5, $\gamma = 100$</code>	0.00598402	3066	0.00139227
<code>fe_degree = 2, $\gamma = 50$</code>	7.25037×10^{-6}	1326	0.000842335
<code>fe_degree = 2, $\gamma = 100$</code>	1.82201×10^{-5}	2388	0.00048086

6.1.4 Problem 0.1, 2D case

	Error at final time	ARKode steps	Last stepsize
Default settings	0.00231121	2277	0.00067853
<code>initial_refinement = 4, $\gamma = 50$</code>	0.000641402	11437	0.000202871
<code>fe_degree = 2, $\gamma = 100$</code>	3.80978×10^{-5}	11559	0.000208635

6.1.5 Problem 0.2, 2D case

	Error at final time	ARKode steps	Last stepsize
Default settings	0.0231104	4403	0.00292041
<code>initial_refinement = 4</code>	0.00612146	9044	0.00161523
<code>initial_refinement = 4, $\gamma = 20$</code>	0.00631621	16450	0.00100964
<code>fe_degree = 2, $\gamma = 20$</code>	5.88644×10^{-5}	8937	0.00179902

Also some tests with `fe_degree = 2` and $\gamma = 50, 100$ were attempted, but were aborted because of the too large amount of work for ARKode: even though the integration continued, the time step was frequently around 10^{-5} , and sometimes an error like this showed up:

```
[ARKode ERROR] arkEvolve
At t = -0.720722, mxstep steps taken before reaching tout.
```

6.1.6 Problem 2.2, 2D case

	Error at final time	ARKode steps	Last stepsize
Default settings	0.00276206	2055	0.000907226
<code>initial_refinement = 4</code>	0.000723828	3167	0.000997511
<code>initial_refinement = 4, $\gamma = 20$</code>	0.000738421	6041	0.000434012

6.1.7 Some words on the 3D case

Things work in the same way in 3D, provided that we choose γ intelligently: in general, compared to the 2D case, it has to be bigger.

However, apart from the difficulty of testing problems with a lot of degrees of freedom (all the tests were done with a 4th generation Intel i5 Core), it seems I may have caught a bug from deal.II, probably related to the way Trilinos objects interact with SUNDIALS. To discuss this, I will refer to the tests I did before implementing the AMG preconditioner, when the program used `SparseMatrix` and `Vector` objects:

- Before switching to Trilinos, solutions (and errors) of problems 0.1 and 0.2 with default settings showed some instability/flickering. Probably the reason was γ being too low, but the program finished its job without complaining. Lowering the initial refinement to 2, while keeping $\gamma = 10$, seemed to improve the situation, especially for problem 0.2.
- Doing the same tests with the Trilinos version leads to an unexpected crash if γ is too low.

Let us stick to problem 0.1. Before Trilinos, some tests results were:

	Error at final time	ARKode steps	Last stepsize
Default settings	0.00424171	990	0.00186179
<code>initial_refinement = 2</code>	0.00827368	630	0.00262032

With Trilinos, the test with `initial_refinement = 2` can still be replicated. Trying with default settings, instead, leads to a crash just after start; setting $\gamma = 50$ leads to the same kind of crash, but at step 57, even though the output until then seems pretty decent:

```
Time step 57 at t = 0.14.
Number of ARKode steps taken so far: 1508
Step size taken on the last internal step: 0.00135621
l2 norm of the error estimator: 0.242684
L2 error on the solution: 0.00300896
```

This is the error message from the program:

This exception – which is used in many places in the library – usually indicates that some condition which the author of the code thought must be satisfied at a certain point in an algorithm, is not fulfilled. An example would be that the first part of an algorithm sorts elements of an array in ascending order, and a second part of the algorithm later encounters an element that is not larger than the previous one.

There is usually not very much you can do if you encounter such an exception since it indicates an error in deal.II, not in your own program. Try to come up with the smallest possible program that still demonstrates the error and contact the deal.II mailing lists with it to obtain help.

A summary of the tests done with the actual version:

	Error at final time	ARKode steps	Last stepsize
Default settings	program crash		
$\gamma < 50$	program crash		
$\gamma = 100$	0.00307315	4585	0.000431185
<code>initial_refinement = 2</code>	0.00789524	631	0.00130313

Some further investigation will be needed.

6.2 Some tests with adaptive refinement

The tests were limited, for simplicity, to the 1D and 2D cases. Some key points:

- Adaptive mesh refinement encounters some troubles in 1D, because we cannot talk about a "face diameter" in dimension 1. Hence, quantities that are resized according to the face diameter may not work as well as in dimension 2 or 3. The main consequence is that we may have to increase γ slightly (a priori) to prevent the numerical solution to explode. For further info, see Issue #12.
- Tweaking the parameters, in particular the tolerance for the Kelly error estimator and the refinement fractions, is important to get a good result. In fact:
 - When a solution tends to return to a flat state, we would like to have a coarser mesh. Hence we have to increase the refinement bottom fraction. Unfortunately, this clashes a bit with requiring the mesh to be finer when necessary, at least with the control on the parameters that we have so far.
 - In a similar fashion, if the refinement bottom fraction is too low, the number of cells tends to grow with time. This may not be optimal, since we could be using more degrees of freedom than actually needed. Fortunately, the problem should not worsen too much because of the maximum level of refinement.

Here animations are important to get a glimpse of what is going on: please refer to the gallery in the README.